
Cyberboswachters

De beste digitale stropers zijn ook de beste cyberboswachters

Dams Tim

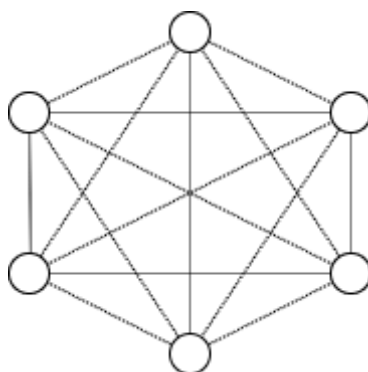


Asymmetrische encryptie

Het probleem met symmetrische encryptie

Wat als je bij symmetrische encryptie met meerdere mensen wilt communiceren zonder dat iedereen elkaars berichten kan zien? Bob kan onmogelijk dezelfde sleutel gebruiken om met Eve te communiceren die hij reeds gebruikte met Alice. Kortom, je hebt per *eindpunt* een aparte sleutel nodig. Het aantal sleutels dat je nodig hebt, zeker als ook alle gebruiker onderling nog eens willen communiceren wordt erg snel erg groot. Je kan dit berekenen met de formule $n * \frac{(n-1)}{2}$ waarbij n het aantal gebruikers voorstelt:

- 6 gebruikers vereisen 15 sleutels.
- 7 gebruikers vereisen 21 sleutels.
- 10 gebruikers vereisen er al 45.
- 100 gebruikers vereisten er 4950!



Figuur 0.1: Bij 6 gebruikers zijn er al 15 sleutels nodig.

Symmetrische encryptie heeft dus een *key distribution problem* wanneer er encryptie op een grote schaal nodig is. Zeker als we spreken over online communicatie, over het internet, wordt de schaal ogenblikkelijk gigantisch groot en wordt het *sleutelmanagement* problematisch. Een andere oplossing is dus aan de orde.

Publieke cryptografie

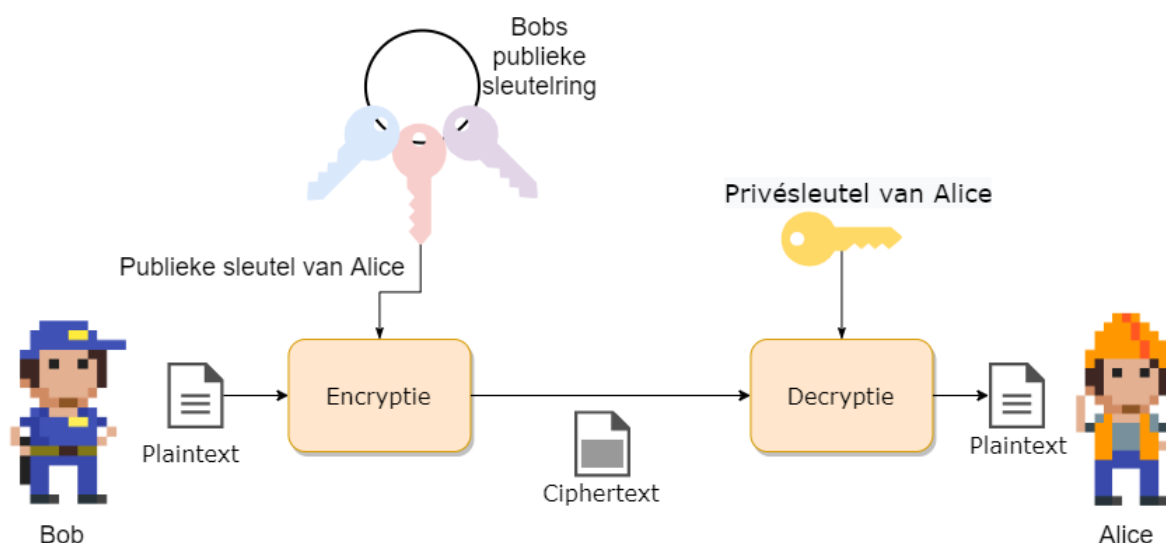
Publieke crypto oftewel **asymmetrische encryptie** zal het probleem met symmetrische encryptie oplossen doordat er gebruikt wordt gemaakt van 2 sleutels:

- 1 publieke sleutel

- 1 private sleutel

De publieke sleutel kan iedereen, vrij, gebruiken om versleutelde berichten mee aan te maken. Echter, enkel de eigenaar van de bijhorende private sleutel zal deze berichten kunnen decrypteren. Kortom, we lossen nu een deel van het sleutelprobleem op: iedereen heeft z'n eigen private sleutel (**en moet deze geheim houden!**) en kan via z'n publieke sleutel berichten krijgen.

De techniek werd in de jaren 70 door Diffie en Hellman ontwikkeld en had een grote impact op de manier waarop beveiligde communicatie op het internet mogelijk maakt.



Je kan publieke encryptie beschouwen als volgt. De publieke sleutel, die niet geheim is, is een openstaande kist. Iedereen kan de kist gebruiken om een geheime boodschap in te plaatsen en vervolgens deze op slot te klikken. Enkel de eigenaar van deze kist heeft de bijpassende private sleutel die echter deze kist kan opendoen en de originele boodschap kan lezen (*decrypteren*).

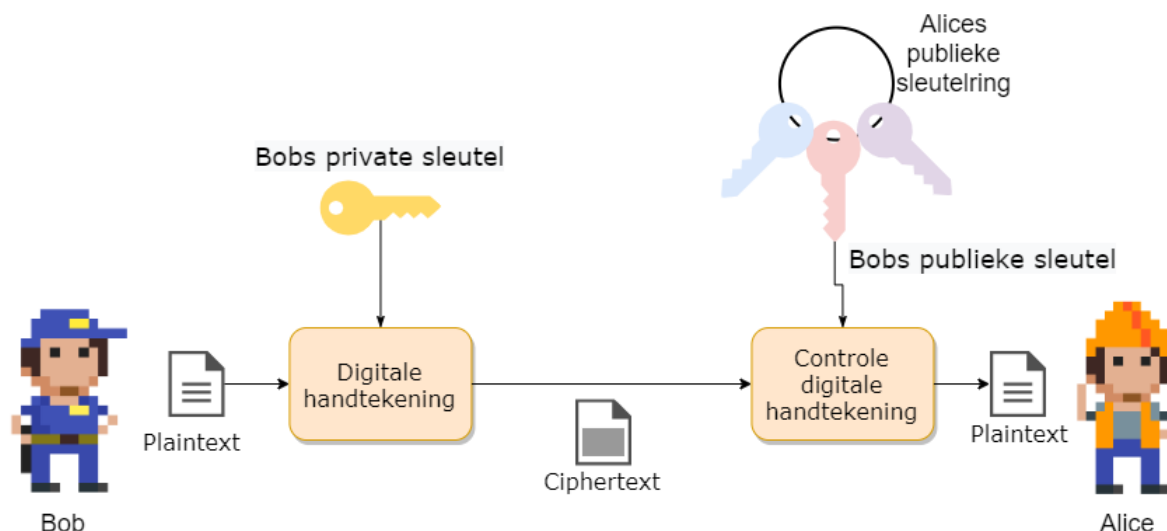


Enkele van de bekendere publieke cryptosystemen zijn onder andere RSA, DSS en El Gamal.

Dualiteit van publieke cryptografie

Asymmetrische crypto zal niet alleen het sleutel-probleem oplossen, het heeft als extra eigenschap dat het publiekprivate sleutelpaar ook dienst kan doen als **digitale handtekening** om te controleren of een boodschap wel degelijk afkomstig is van een specifiek persoon. Hierbij zal een private sleutel gebruikt

worden om het digitale bericht te ondertekenen. Daar de ondertekenaar de enige persoon kan zijn die deze private sleutel in z'n bezit heeft, kan men zijn identiteit bevestigen door de bijhorende publieke sleutel te gebruiken. Enkel de bijhorende publieke sleutel zal hiervoor gebruikt kunnen worden en zo hebben we een vorm van integriteit of data authenticatie.



We zullen dit concept verderop uitwerken, maar eerst gaan we bekijken hoe publieke crypto juist werkt.

Diffie-Hellman sleutel uitwisseling

Dankzij public crypto hebben we nu een systeem om sleutels op een veilige manier uit te wisselen. Het is namelijk zo dat symmetrische crypto sneller is én dus voor (realtime) communicatie interessanter is. We weten echter dat het sleutelmanagement bij symmetric crypto een probleem is als we met grote groepen gebruikers zitten. Het Diffie-Hellman sleuteluitwisselingsconcept helpt ons hierbij: het laat toe dat twee gebruikers sleutels over een onveilig kanaal kunnen uitwisselen op een veilige manier.

Zowel Bob als Alice genereren eerst een publiek/privaat sleutelpaar dat ze voor deze sessie wensen gebruiken ter communicatie. Vervolgens stuurt ieder z'n publieke sleutel naar de ander. De ontvanger zal deze publieke sleutel combineren met de eigen private sleutel wat zal resulteren in een nieuw *shared secret* dat beide nu kennen en kunnen gebruiken, bijvoorbeeld, als de symmetrische sleutel om verdere communicatie te bestendigen.

De reden dat dit werkt is met dank aan de modulo operator en de eigenschappen ervan. Een voorbeeld:

	Alice	Bob
Stap 1	Alice kiest een geheim getal A. Ze kiest $A = 3$	Bob kiest ook een geheim getal $B = 6$.
Stap 2	Alice berekent $7^A \% 11 \Rightarrow 3^3 \% 11 = 2$, genaamd X	Bob berekent $7^B \% 11 \Rightarrow 7^6 \% 11 = 4$, genaamd Y
Stap 3	Alice stuurt 2 naar B	Bob stuurt 4 naar Alice
Stap 4	Alice berekent $X^A \% 11 = 2^3 \% 11 = 9$	Bob berekent $Y^B \% 11 = 4^6 \% 11 = 9$

Zoals je merkt kunnen nu Alice en Bob het berekende getal 9 als gedeeld geheim kennen. Enkel zij 2 kennen dit getal.



Uiteraard zullen in de praktijk Bob en Alice véél grotere getallen kiezen dan 3 en 6.

RSA

Eén van de oudste, maar nog steeds populairste, publieke cryptosystemen is het in 1977 ontwikkelde RSA algorithm. RSA, wat staat voor de achternamen van de 3 ontwikkelaars (Rivest, Shamir en Adleman) gebruikt sleutels van 1536 tot 4096 bits lang. Het systeem is vrij traag maar heeft natuurlijk als voordeel dat het veilige sleuteltransmissie toestaat over een onveilig kanaal: we zien daarom vaak RSA gebruikt worden om eerst sessiesleutels uit te wisselen, vervolgens wordt overgeschakeld op een sneller symmetrisch cipher.

De exacte berekeningen die gebeuren tijdens encryptie en decryptie nemen ons iets te ver, maar volgend voorbeeld toont een vereenvoudigde wijze waarop RSA wordt toegepast:

Data encrypteren met behulp van asymmetrische versleuteling gebeurt op bijna dezelfde wijze als de Diffie-Hellman sleutel uitwisseling. Ook nu zullen beide zijde rekenen op de eigenschappen van de modulo-operator om over een onveilig kanaal veilige communicatie te kunnen doen.

Eerst dient een publieke sleutel aangemaakt te worden. Hiertoe dient Bob 2 grote priemgetallen, p en q te kiezen, bijvoorbeeld $p=17$ en $q=11$. Vervolgens berekent Bob N door deze priemgetallen met elkaar te vermenigvuldigen ($p \cdot q$ geeft $17 \cdot 11$, $N=187$). Bob kiest nu nog een priemgetal e , bijvoorbeeld 7. Bob kan nu zijn eigen geheime, private sleutel d maken, namelijk $e \cdot d = 1 \% ((p-1) \cdot (q-1))$ wat

dus $7 * d = 1\%(16 * 10)$ geeft of $7 * d = 1\%160$. Om nu d te vinden moeten we een getal vinden zodat $7 * d$ een veelvoud van $1\%160$ geeft, dus bijvoorbeeld 1, 161, etc. In dit geval vinden we $d=23$.



Om d te berekenen maken we gebruik van de zogenaamde *Uitgebreid Euclidisch algoritme*, een eeuwenoud algoritme gebaseerd op het *Algoritme van Euclides* dat we in het lager leerden gebruiken om de grootste gemene deler te berekenen van 2 getallen.

Bob heeft dus nu:

- Publieke sleutel bestaande uit $N = 187$ en $e = 7$.
- Private sleutel $d = 23$.

Iedereen die nu naar Bob iets wilt sturen kan dit via z'n publieke sleutel (N en e). Stel dat Alice het ascii-karakter X naar Bob wil sturen. De ascii-waarde van X is 88. De encryptie door Alice gebeurt dan als volgt: $C = data^e \% N$. De te versturen ciphertext C wordt dus: $(88^7)\%187$ oftewel $C=11$.

Enkel Bob zal deze ciphertext met zijn private sleutel d kunnen decrypteren door $C^d(\%N)$ te doen, oftewel $11^{23}\%187$ wat terug de plaintext 88 geeft!



de sterkte van publieke crypto stoelt dus op het feit dat ontbinden van (grote) getallen in factoren computationeel veel moeilijker is dan de omgekeerde stap, namelijk 2 getallen met elkaar vermenigvuldigen.

15621 in z'n factoren ontbinden is veel moeilijker dan de getallen 123 en 127 vermenigvuldigen (wat dus ook 15621 zal geven).

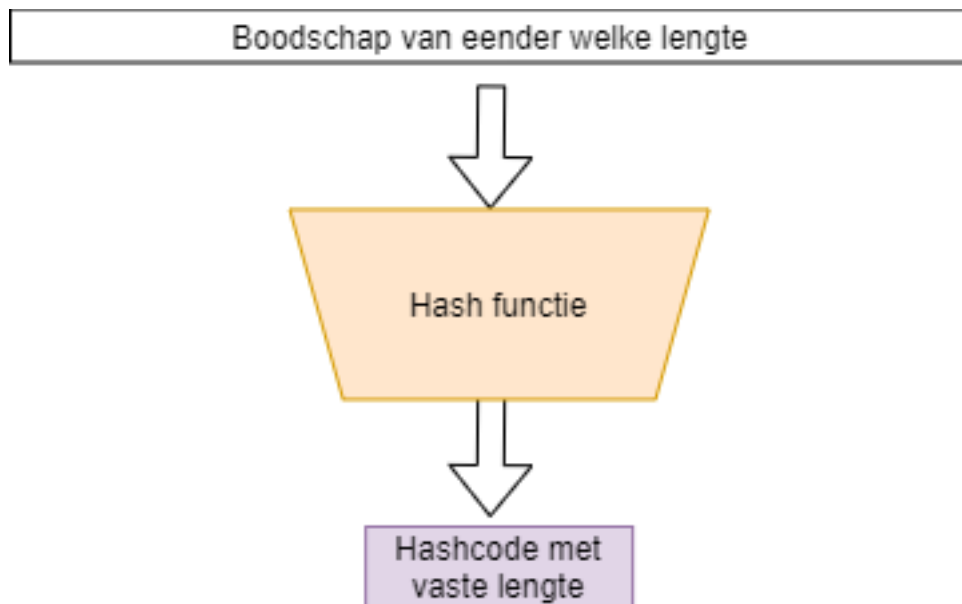


Zonder in detail te treden hoe cryptocurrencies en blockchains werken, is het nuttig om te vermelden dat bij cryptocurrencies ook de public crypto concepten worden gebruikt. Ook hier is je private sleutel uiterst belangrijk: enkel de eigenaar van de private sleutel “bezit” de bijhorende cryptocurrencies in de chain. Daarom is het uiterst belangrijk dat je NOOIT je private sleutel aan derden geeft, want zo geef je hen toegang tot jouw coins en kunnen ze vervolgens deze stelen door de private sleutel te vervangen.

Intermezzo: Hashes

Een zogenaamde hash is een concept uit de informatica die we gebruiken om te controleren of een digitaal stuk tekst werd aangepast of niet. Door de tekst in een hashfunctie te steken wordt een hash aangemaakt. Deze hash is een stuk code met een vaste lengte, ongeacht de originele input. Wanneer

1 bit of meer wordt aangepast in de originele boodschap dan zal deze in een totaal andere hash resulteren. Enkel dus wanneer een identiek stuk tekst als invoer (tot op bitniveau identiek) wordt gebruikt zullen twee hashen gelijk zijn.



Voorgaande is uiteraard onmogelijk: daar een hash meestal veel korter is dan de originele boodschap, is het mathematisch mogelijk dat 2 totaal verschillende teksten toch dezelfde hash geven. Het is de opdracht van een goede hashfunctie om dit soort **hash collisions** zo klein mogelijk te houden!

Een hash-functie is niet omkeerbaar: men mag onmogelijk aan de hand van een hash (ook wel *digest* of *hashcode* genoemd) terug de originele tekst kunnen achterhalen. Een hashfunctie is dus een eenrichtingsfunctie, ook wel afbeelding genoemd in wiskundige termen.

Er bestaan veel verschillende hash-functies. Enkele van de bekendere zijn: * MD5, oftewel Message Digest 5: deze zal een 128-bit hashwaarde genereren. * SHA-X, oftewel Secure Hash Algorithms. Zo is er SHA-256 wat een 256 bits hash zal genereren.

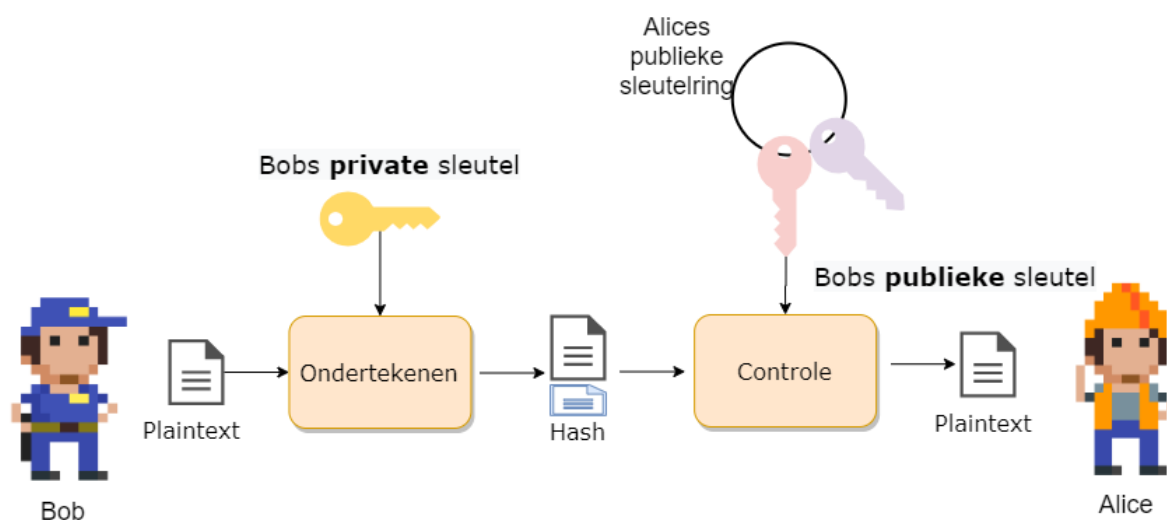
De sterkte van een hash algoritme zit hem in de grootte van de kans waarop hash collisions kunnen optreden. Zo zijn er bij MD5 al veel meer collisions gevonden dan bijvoorbeeld bij het recenter gepubliceerde SHA3-512 algoritme.

Boodschappen ondertekenen

Een probleem bij online communicatie is dat we geen zekerheid hebben dat het ontvangen bericht wel degelijk van de persoon komt van wie we verwachtten dat deze het bericht had opgesteld. We kunnen daarom het publieke crypto systeem gebruiken om berichten te ondertekenen. Daar enkel Bob de bijhorende private sleutel kan hebben die bij z'n publieke sleutel hoort, is het bezit van deze private sleutel hebben het bewijs dat hij de rechtmatige eigenaar van een bepaalde publieke sleutel is.

Onder andere RSA laat toe om een digitale handtekening (**digital signature**) te plaatsen bij een bericht om zo te bewijzen dat de verzender de rechtmatige eigenaar van een bijhorende publieke sleutel is.

Een digitale handtekening wordt als extra bericht achteraan de te versturen boodschap geplaatst. De signature is als het ware een hash berekend aan de hand van de private sleutel. De ontvanger zal nu met de bijhorende publieke sleutel van de verzender kunnen verifiëren of de bijhorende private sleutel werd gebruikt om de handtekening te genereren.



Om een digitale handtekening te berekenen moeten we eerst een hash van het bericht berekenen. We gebruiken hier bijvoorbeeld MD5 of één van de SHA-algorithmes voor. Deze hash gaan we nu “verpakken” met de private sleutel.

Stel dat de hash 35 is en we hebben volgende sleutelpaar:

- publieke sleutel: $e=5$ en $n=91$
- private sleutel: $d=29$.

De handtekening wordt berekend door: $s = m^d \% n$ oftewel $s = 35^{29} \% 91$ wat 42 geeft.

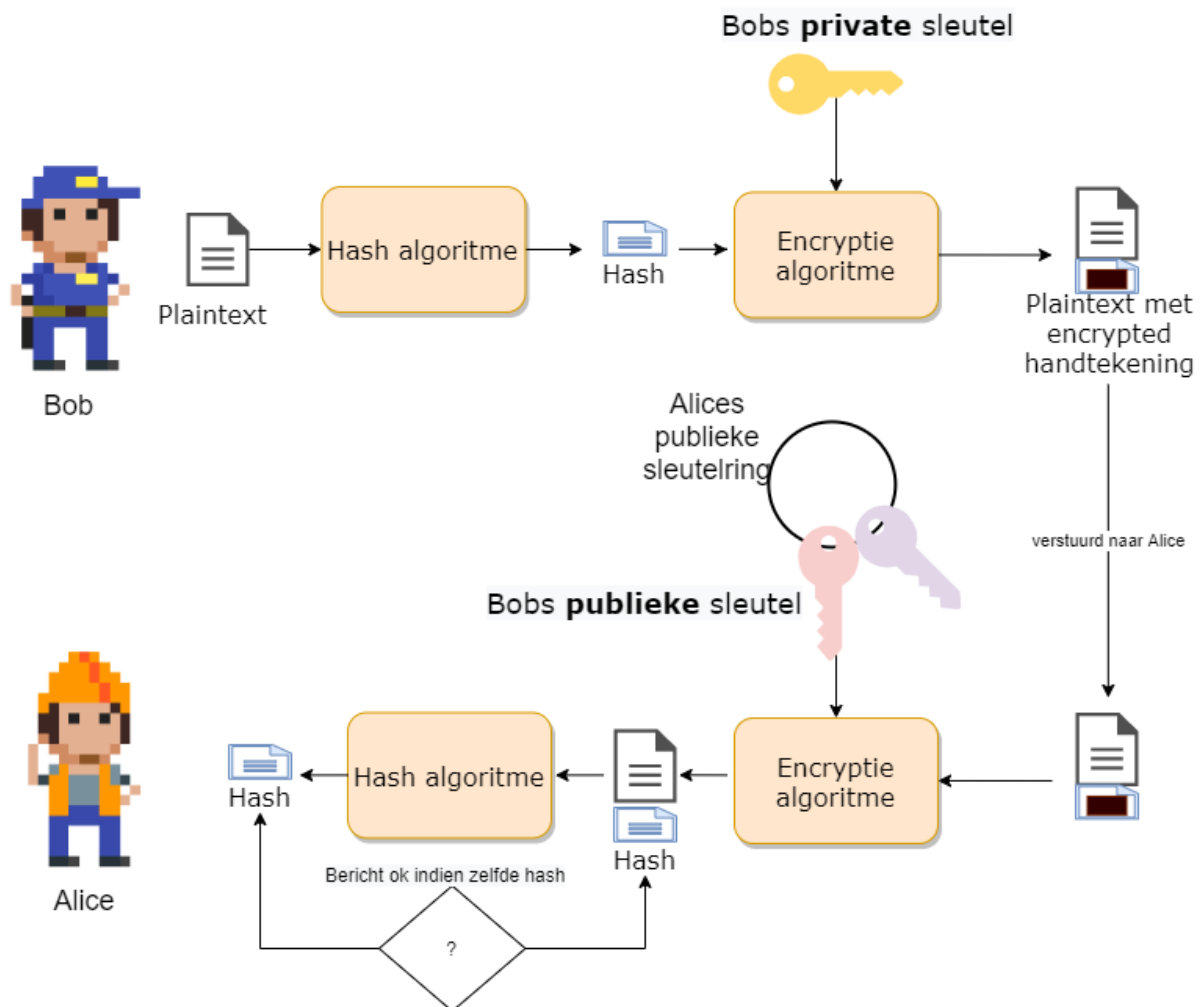
We versturen dus naar de ontvanger de boodschap zelf en de bijhorende handtekening: 35 , 42.

De ontvanger kan nu controleren of de ontvangen boodschap 35 dezelfde handtekening geeft. Hij

gebruikt hiervoor de publieke sleutel e en berekent: $42^e == 35^n$, als dit overeenkomt dan weet de ontvanger dat hij het bericht kan vertrouwen.



Getallen voorbeeld komt van hier

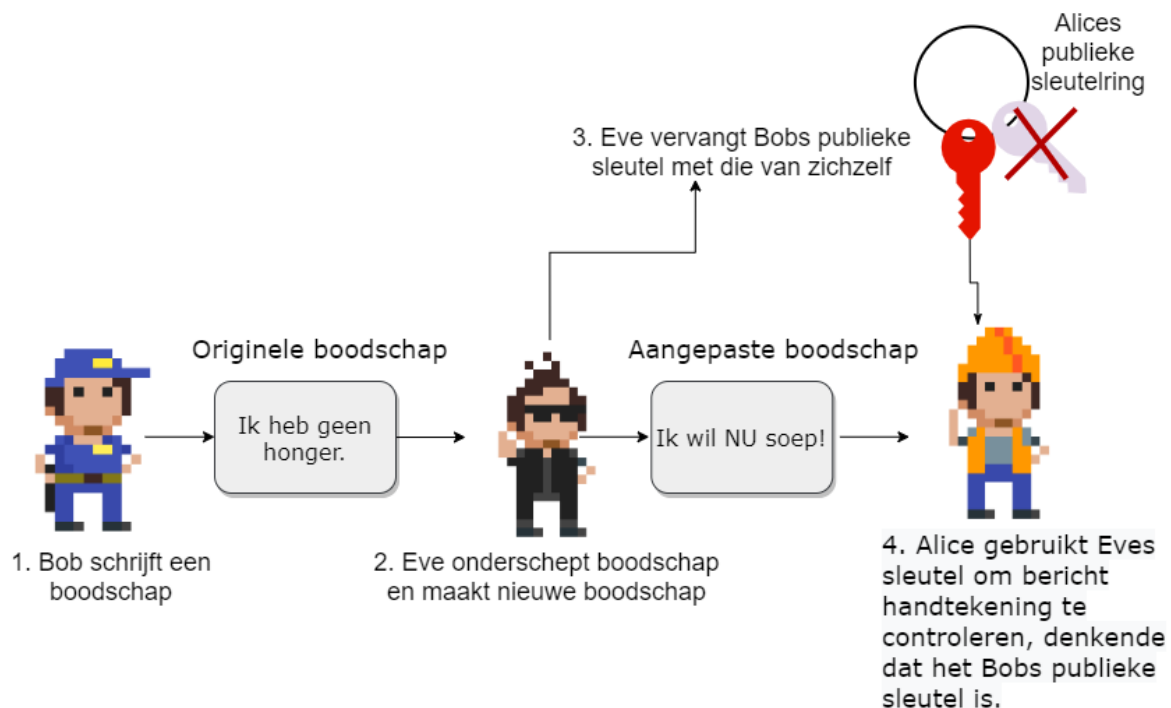


Het probleem met digitale handtekeningen

We hebben echter een probleem. Hoe weet je eigenlijk dat je wel de juiste publieke sleutel gebruikt. Publieke sleutels zijn, wel, publiek. Iedereen kan jou een publieke sleutel geven en zeggen “Dit is de sleutel van persoon X” zonder dat jij kan controleren of dat zo is.

We kunnen daarom als kwaadwillig persoon bijvoorbeeld een legaal bericht onderscheppen, aanpassen

en dan vervolgens ondertekenen met onze eigen handtekening. Als we vervolgens aan de ontvanger kunnen wijsmaken dat jouw publieke sleutel zagezegd bij de originele verzender hoort, dan zal de ontvanger jouw aangepaste bericht “geloven”.



Kortom, we hebben een manier nodig om de **identiteit van de eigenaar** van een publieke sleutel te verifiëren. Kom binnen: **certificaten**.

Digitale certificaten

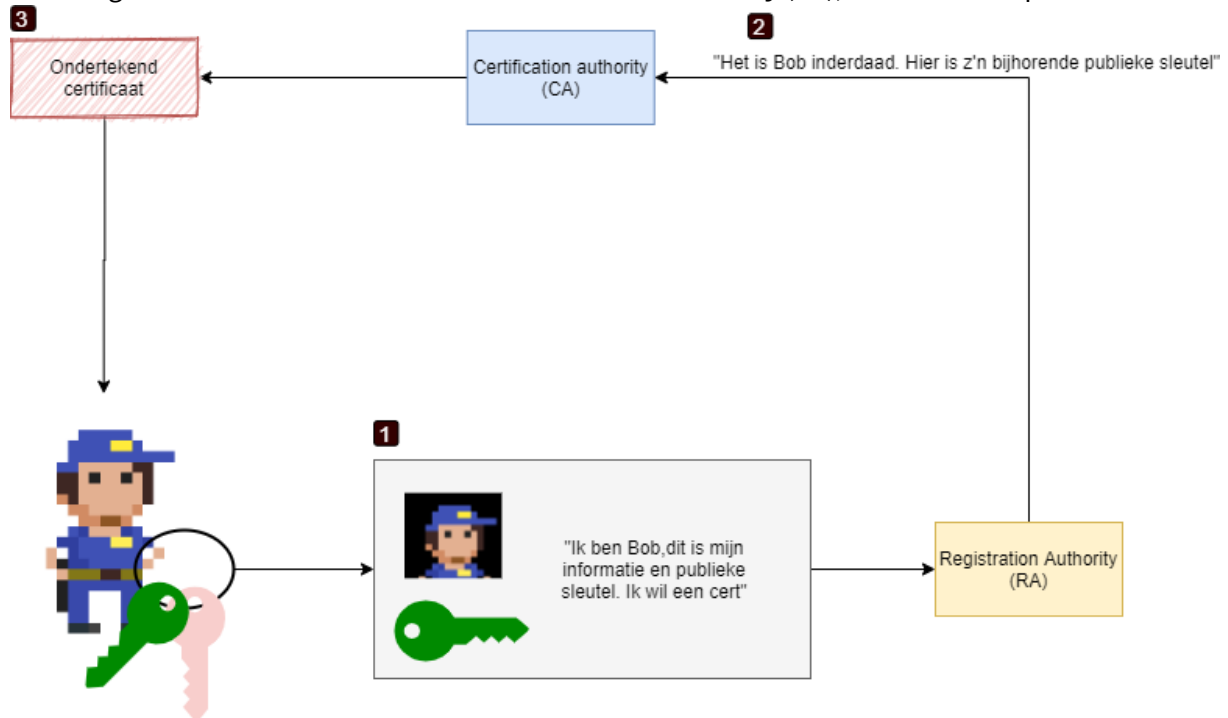
Een certificaat is een (digitaal) document dat de identiteit van een gebruiker bindt aan een publiek sleutel. Dit document werd digitaal ondertekent door een vertrouwde derde partij (**trusted third party**) zodat bij twijfel van de echtheid van het certificaat men altijd bij deze derde partij terecht kan. Uiteraard is het belangrijk dat we deze derde partij kunnen vertrouwen, anders kunnen we ook niet de het certificaat vertrouwen dat zijn onderschrijven.



Certificaten worden beschreven in de **X.509** standaard.

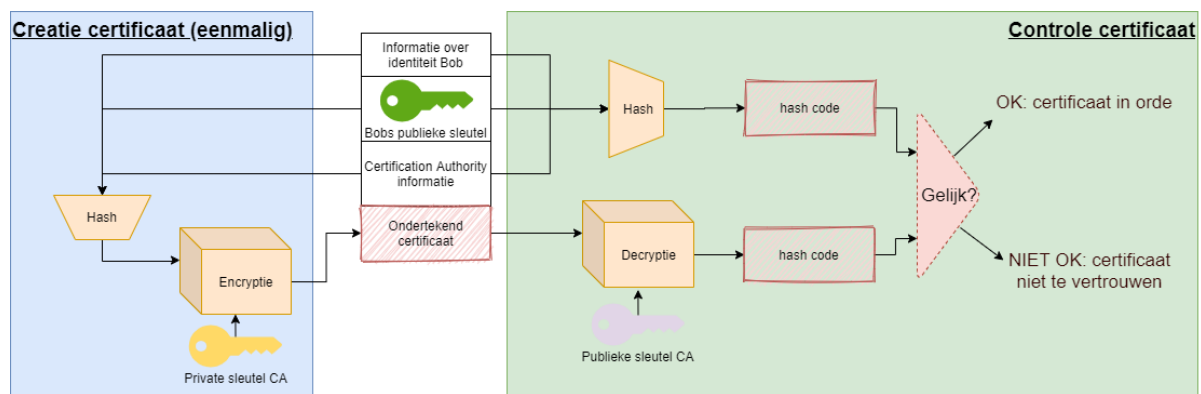
Om een certificaat aan te maken dient Bob naar een **Registration authority** (RA) gaan die zijn identiteit zal verifiëren. Dit gebeurt aan de hand van de typische documenten die ook buiten het Internet worden gebruik om iemands identiteit te bewijzen: identiteitskaart, paspoort, rijbewijs,

etc. In sommige gevallen zal de RA zelfs eisen dat Bob zich naar een fysiek kantoor begeeft om daar z'n identiteit *in te flesh* te bewijzen. Indien de RA de identiteit heeft bevestigd zal deze de aanvraag van Bob doorsturen naar een **Certification authority (CA)**, inclusief Bobs publieke sleutel.

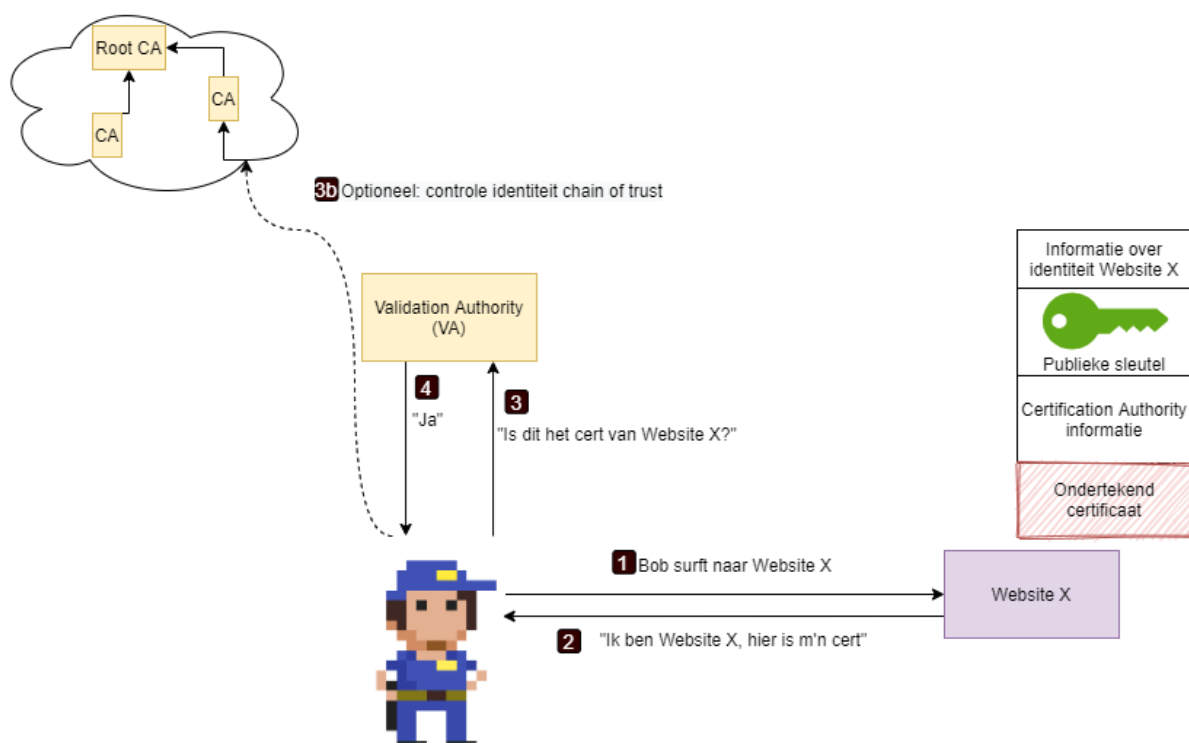


Het gehele systeem van CA's, RA's, etc. dat bestaat om certificaten uit te geven, beheren en bewijzen heet een **public key infrastructure (PKI)**.

De CA zal deze informatie gebruiken om een certificaat, van een bepaalde levensduur, te genereren. Hierbij zal de echtheid van de CA achter af bewezen kunnen worden door de CA: * Het certificaat is een geëncrypteerde hash van Bobs publieke sleutel, informatie over de CA en over Bob. De encryptie van de hash gebeurt aan de hand van de private sleutel van de CA. * Om later de echtheid van een certificaat te testen voldoet het om dezelfde hash te genereren (publieke sleutel, info over Bob en CA) en deze te vergelijken met het certificaat na decryptie met de publieke sleutel van de CA. Als deze gelijk zijn weten we dat het certificaat door de gegeven CA werd ondertekend (enkel hun private/publiek sleutel paar zal terug de originele hash geven).

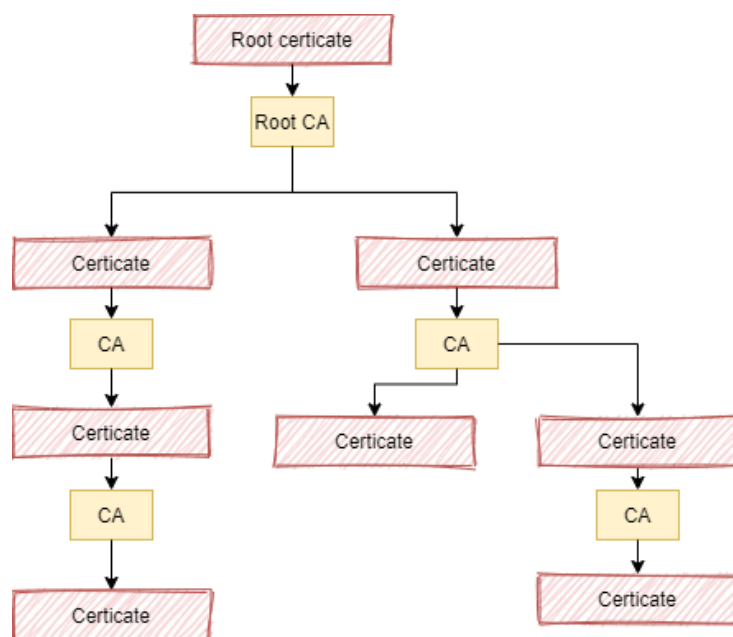


Voorgaande proces zal plaatsvinden wanneer je browser via een **https** verbinding surft naar een website en zo wil controleren of wel degelijk met de website wordt gecommuniceerd en niet met een imposter. Indien de browser (of de gebruiker) twijfelt aan de echtheid van de publieke sleutel van de CA die het certificaat van de website ondertekent, dan zla voorgaande proces zich herhalen, maar deze keer om het certificaat van de CA te controleren met behulp van een bovenliggende CA. Op die manier kan het dus zijn dat een keten van CA's ontstaan die telkens CA's onder zich bewijzen. Uiteraard zal er steeds bovenaan zo'n ketting een **root CA** staan. Als je die vertrouwt, dan kan je al de CA's er onder dus ook vertrouwen... maar ook vice versa!



Het ergste voor een CA dat kan voorvallen is dat de betrouwbaarheid van de CA in het gedrang komt.

Als een CA bijvoorbeeld weet heeft van een potentiële inbraak op hun systemen dan bestaat er de kans dat aanvallers de private sleutel van de de CA hebben bemachtigd en dus zelf certificaten *op naam van de CA* kunnen genereren, met alle gevolgen van dien! Indien dus deze kan bestaat dan is er een *breach of trust* en zullen alle certificaten van deze CA als ongeldig worden bestempeld, inclusief alle certificaten van sub-CA's! Dit kan verregaande gevolgen hebben.



Certificaten bekijken

In iedere moderne browser kan je snel bekijken hoe zo'n certificaat er juist uit ziet. Als je via een https verbinding naar een website surft dan op het slotje naast de URL n de adresbalk klikt kan je doorklikken om het certificaat te openen. Als je naar <https://www.belgium.be> surft en dit doet dan krijg je eerst wat samenvattende informatie:

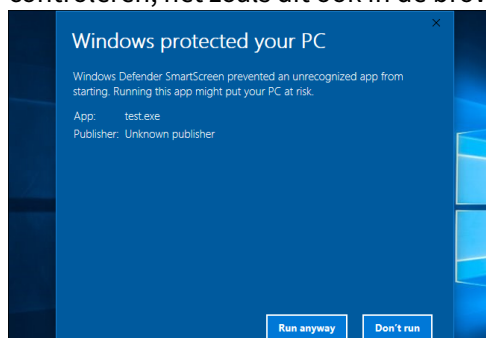
Zo zien we onder andere de geldigheidsduur, alsook de CA die dit certificaat heeft gegenereerd. Onder details kunnen we onder andere de publieke sleutel zien van de website alsook de gebruikte algorithmes voor de hash, e.d.

En op de laatste tab, Certificeringspad, zien we de chain of trust. We kunnen vervolgens hier de bovenliggende certificaten bekijken.



Naast certificaten voor webserver (zogenaamde **SSL certificaten**) kan je ook een persoonlijk certificaat aankopen om je eigen identiteit aan derden te bewijzen tijdens bijvoorbeeld email-communicatie. Voorts heb je ook **code signing** certificaten die de echtheid van een applicatie bewijzen zodat je zeker bent dat je geen malware installeert als je programma X hebt gedownload.

Als je in Windows 10 een applicatie of installer probeert uit te voeren dan zal de ingebouwde *SmartScreen* service ogenblikkelijk de echtheid (of ontbreken van) het certificaat controleren, net zoals dit ook in de browser zou gebeuren.



Dankwoord

Een oprechte dank aan Stefaan Somerling (RealDolmen) voor de feedback op het hoofdstuk omtrent GDPR. Geef gerust een sein als u, conform de GDPR wetgeving, liever uw naam niet in dit document ziet staan ;)

